

Closing the Loop for Software Remodularisation - REARRANGE: An Effort Estimation Approach for Software Clustering-based Remodularisation

Alvin Jian Jia Tan
School of Information Technology
Monash University Malaysia
Malaysia
alvin.tan@monash.edu

Chun Yong Chong
School of Information Technology
Monash University Malaysia
Malaysia
chong.chunyong@monash.edu

Aldeida Aleti
Faculty of Information Technology
Monash University
Australia
aldeida.aleti@monash.edu

Abstract—Software remodularisation through clustering is a common practice to improve internal software quality. However, the true benefit of software clustering is only realized if developers follow through with the recommended refactoring suggestions, which can be complex and time-consuming. Simply producing clustering results is not enough to realize the benefits of remodularisation. For the recommended refactoring operations to have an impact, developers must follow through with them. However, this is often a difficult task due to certain refactoring operations' complexity and time-consuming nature.

Index Terms—effort estimation, software remodularisation, software clustering, refactoring

I. INTRODUCTION

In the work by Martini [1], the authors discussed that when 42 developer work months (DWM) were spent on refactoring, the effort spent on maintenance was reduced by 53.34 DWM, demonstrating a quantifiable benefit of refactoring. Ensuring high modularity pays off in the long term (from the perspective of development and maintenance effort), quantifying the business value of modularization. However, developers might be hesitant to follow the suggested refactoring operations due to the uncertain amount of effort needed, especially when the suggestions are “Big Bang” remodularisation (refactoring operations that reorganize most of the system's classes into packages). This work addresses this issue by introducing an approach that closes the loop in existing software clustering and remodularization research [2]–[4]. The proposed approach estimates the time required to carry out suggested refactoring operations based on the past evolutionary history of the analyzed software. By informing developers of potentially complicated and time-consuming refactoring operations, the approach helps developers prioritize refactoring efforts and plan accordingly. This allows practitioners to make more informed decisions about refactoring operations within time and budget constraints.

II. PROPOSED APPROACH

The proposed approach tries to **predict the effort required in person-hours to carry out the suggested refactoring operations given by the software clustering results** based

on past refactoring operations performed by the developers, which involves four main parts, depicted in Figure 1.

1) *Software Systems*: The software systems¹ used in this study are chosen pseudo-randomly, with a mixture of GitHub projects and Apache projects to provide a meaningful representation of different types of open-source projects with various sizes and complexities.

2) *Refactoring Effort Indicator*: A proxy effort is then generated to estimate the refactoring effort [5]. Taking Apache Camel project and one of its commits as an example, Total Commit Time (TCT) = 9 person-hours, Refactored Lines of Code ($RLoC$) = 25 lines, Commit Lines of Code ($CLoC$) = 102 lines. The Refactoring Time Taken (RTT) for this commit is given as $9 \times \frac{25}{102} = 2.205$ person-hours.

3) *Data Extraction of Features*: **Software Features** - for each refactoring operation in the analyzed project, the software features of the classes involved in the refactoring operation are extracted using the Java CK (Chidamber and Kemerer) code metrics calculator [6]. **Refactoring Features** - we use Refactoring Miner [7] to detect refactoring operations applied throughout the evolution of a project. Refactoring Miner records all refactoring operations between each minor version of the project, from the first to the last release. **Dependency Features** - we employ *Depends* [8] to extract dependencies between classes in the analyzed software. *Depends* gathers syntactical relations among source code entities, such as files and methods, and extracts different types of dependencies from a pair of classes, such as import, contain, call, return, and implement.

4) *Training the Machine Learning Models*: H2O AutoML [9] is used to learn the relationship between features and the proxy effort. In our work, the Gradient Boosting Machine method performs the best as a regression model to predict refactoring effort.

III. RESULTS

The MAE (Mean Absolute Error) of 5.47 for REARRANGE is within an acceptable error of 1 working day of

¹<https://figshare.com/s/0d24fa70b21104e7dcf1>

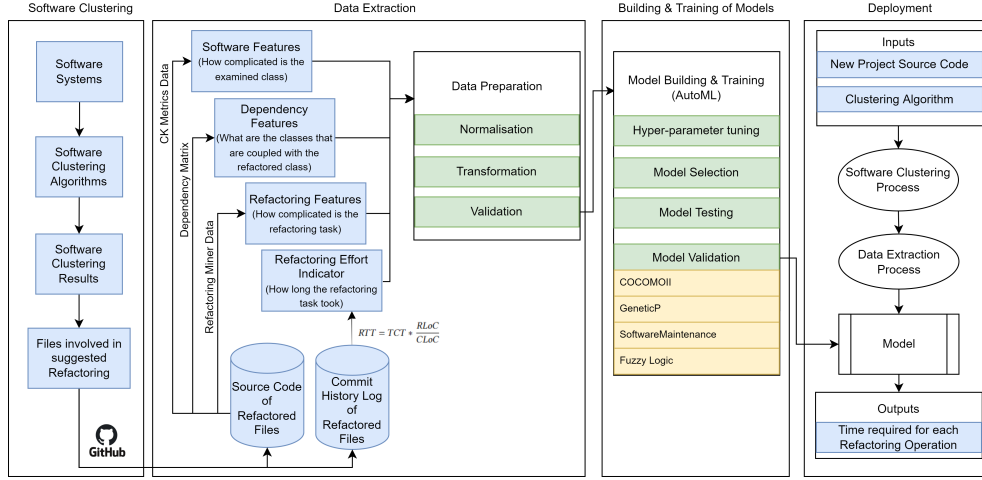


Fig. 1: Overall workflow of REARRANGE.

8 person-hours. This step answers the first research question, *RQ1: How effective is REARRANGE in predicting the effort required for implementing the clustering results?* By comparing REARRANGE against sanity checks, we answer the second research question, *RQ2: Does REARRANGE satisfy the requirements of a baseline estimation model?* Finally, we compare REARRANGE against other software cost estimation models as shown in Table I. By comparing MAE between REARRANGE (5.47) and Genetic Estimation (453.31) as the next best alternative from Table I, it is evident that REARRANGE performs better at estimating refactoring effort. The significant difference in MAE is because we compare REARRANGE to a software effort estimation model. The lack of a direct comparison further highlights the need for a refactoring effort-focused estimation model. The RMSE results show that the existing software cost estimation models are not good estimators of the refactoring effort, answering the third research question, *RQ3: How does REARRANGE compare to other software effort estimation models?*

TABLE I: Benchmark for REARRANGE (person-hour)

Estimation Model	R2	RMSE	MAE
Mean	-0.974	21.37	18.93
REARRANGE	0.538	15.38	5.47
COCOMOII	-116903	5201.57	893.22
GeneticP	-21469.5	2229.15	453.31

Although REARRANGE may sometimes overestimate the time required for smaller refactoring operations, it can accurately identify and predict large and complex refactoring operations that are incredibly disruptive to developers. Similarly, existing software estimation models significantly overestimate the time required compared to REARRANGE, resulting in high RMSE and MAE values. Developer feedback was then obtained on the proposed refactoring operations and the estimated efforts of the chosen test subjects. This is done by filing issue reports on the relevant GitHub repository or Jira. Among the total 53 suggested refactoring operations and its estimated effort required, 23 received a positive response with

only one negative response. Overall, the positive sentiments from developers show that REARRANGE is able to provide reasonable estimates that are relevant to software developers. Table II shows sample responses from developers.

TABLE II: Results from Developer Validation

Project Name	Issue ID	No. of Refactoring	Positive	Negative	Status
Dropwizard	5402	4	0	0	Pending
Redisson	4363	5	0	0	Pending
Okhttp	7355	17	17	0	Closed
Java-tron	4500	2	1	1	Closed

REFERENCES

- [1] A. Martini, E. Sikander, and N. Medlani, "Estimating and quantifying the benefits of refactoring to improve a component modularity: a case study," in *2016 42th Euromicro conference on software engineering and advanced applications (SEAA)*. IEEE, 2016, pp. 92–99.
- [2] C. Y. Chong and S. P. Lee, "Automatic clustering constraints derivation from object-oriented software using weighted complex network with graph theory analysis," *Journal of Systems and Software*, vol. 133, pp. 28–53, 2017.
- [3] A. J. J. Tan, C. Y. Chong, and A. Aleti, "E-sc4r: Explaining software clustering for remodularisation," *Journal of Systems and Software*, vol. 186, p. 111162, 2022.
- [4] C. Y. Chong and S. P. Lee, "Constrained agglomerative hierarchical software clustering with hard and soft constraints," in *2015 International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)*. IEEE, 2015, pp. 177–188.
- [5] C. Schröder, A. van der Feltz, A. Panichella, and M. Aniche, "Search-based software re-modularization: a case study at adyen," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2021, pp. 81–90.
- [6] M. Aniche, *Java code metrics calculator (CK)*, 2015, available in <https://github.com/mauricioaniche/ck>.
- [7] N. Tsantalis, A. Ketkar, and D. Dig, "Refactoringminer 2.0," *IEEE Transactions on Software Engineering*, vol. 48, no. 03, pp. 930–950, mar 2022.
- [8] W. Jin, Y. Cai, R. Kazman, Q. Zheng, D. Cui, and T. Liu, "Enre a tool framework for extensible entity relation extraction," in *Proceedings of the 41st International Conference on Software Engineering Companion Proceedings*. IEEE Press, 2019, pp. 67–70.
- [9] E. LeDell and S. Poirier, "H2O AutoML: Scalable automatic machine learning," *7th ICML Workshop on Automated Machine Learning (AutoML)*, July 2020.